



Πανεπιστήμιο Δυτικής Αττικής

Τμήμα Μηχανικών Πληροφορικής και Υπολογιστών

ΜΕΤΑΓΛΩΤΤΙΣΤΕΣ

Εργαστηριακή Εργασία Β

«Bison και Uniclips»

ΟΝΟΜΑΤΑ / ΑΜ:

- Ξηρουδάκης Παναγιώτης (ΠΑΔΑ-18390217)
- Μάθησης Ιωάννης Ιάσων (ΠΑΔΑ-18390042)
- Λουκάς Στυλιανός (ΠΑΔΑ-22390126)
- Σοφιανίδης Κωνσταντίνος (ΠΑΔΑ-22390210)
- Τσαρτσίου Ερνίστ (ΠΑΔΑ-22390233)

ΟΜΑΔΑ 1 ΤΜΗΜΑ Α4

2024 - 2025

Περιεχόμενα

Εισαγωγή	3
Τεκμηρίωση του Λεκτικού αναλυτή.....	3
Τεκμηρίωση του Συντακτικού Αναλυτή και της σχεδίασής του.....	6
Τρόπος μεταγλώττισης και τρεξίματος.....	8
Εξαντλητικοί έλεγχοι.....	8
Τεκμηρίωση in.txt.....	12
Τεκμηρίωση in_wrong.txt.....	17
Αναφορές κώδικα	19
Αναλυτική αναφορά αρμοδιοτήτων ομάδας.....	20

Εισαγωγή

Η παρούσα εργασία περιλαμβάνει την τεκμηρίωση της υλοποίησης ενός λεκτικού και συντακτικού αναλυτή (lexer και parser) για την γλώσσα UNI-CLIPS. Ο λεκτικός αναλυτής έχει σχεδιαστεί για να αναγνωρίζει tokens και υποστηρίζει την ανίχνευση λαθών. Ο συντακτικός αναλυτής καθορίζει τη γραμματική της γλώσσας και αναλύει τη δομή των δηλώσεων με βάση προκαθορισμένους κανόνες. Το σύστημα υπολογίζει τον συνολικό αριθμό των σωστών και λανθασμένων λέξεων, καθώς και των εισόδων που αναλύθηκαν, και παράγει συνολικό απολογισμό με μηνύματα επιτυχίας ή αποτυχίας.

Τεκμηρίωση του Λεκτικού αναλυτή

Η λεκτική ανάλυση όπως έχει υλοποιηθεί στο αρχείο flex, είναι σχεδιασμένη με τέτοιο τρόπο ώστε να διαχωρίζει τα στοιχεία της γλώσσας και να αναγνωρίζει τόσο τα προκαθορισμένα tokens όσο και την ύπαρξη ορθογραφικών λαθών. Σημαντικό είναι η πρόβλεψη για κατηγοριοποίηση των tokens ανάλογα με το αν είναι έγκυρα ή μη, ώστε να εντοπίζεται η καταγραφή της εγκυρότητας της εισόδου. Αυτό επιτυγχάνεται με την καταμέτρηση σωστών (correctWords) και λανθασμένων (incorrectWords) λέξεων. Λέξεις στην υλοποίηση μας θεωρούνται οι αριθμητικοί τελεστές, το ίσον, το βέλος (->), οι ακέραιοι και πραγματικοί αριθμοί, οι μεταβλητές, οι πρωταρχικές συναρτήσεις deffacts, defrule, test, printout t, read και bind, τα ονόματα ορισμών και στοιχείων γεγονότων καθώς και οι συμβολοσειρές. Η ενσωμάτωση ενός error state (<error>) γίνεται για να μεταβαίνουμε όταν έχουμε λάθος είσοδο (τα σύμβολα-token που δίνει δεν ανήκουν στην γλώσσα) από τον χρήστη. Σε αυτήν την κατάσταση μένουμε μέχρι να λάβουμε κάποιον διαχωριστή που σημαίνει ότι έχει τελειώσει το εκάστοτε token και επιστρέφουμε στην εκτέλεση του προγράμματος.

Στην αρχή του αρχείου teliko.l δηλώνεται η οδηγία %option noyywrap, έτσι ώστε να γνωρίζει ο λεκτικός αναλυτής πως η ανάλυση τελειώνει μετά το πρώτο αρχείο εισόδου.

Έπειτα ακολουθεί η δήλωση της κατάστασης error η οποία χρησιμοποιείται στο πεδίο των κανόνων.

Στο πεδίο ανάμεσα στα {% και %} υπάρχει ο κώδικας C, που εισάγει τα απαραίτητα header αρχεία για τις συναρτήσεις που θα χρησιμοποιηθούν καθώς και του αρχείου teliko_y.h το οποίο παράγει ο bison και χρησιμοποιείται για την σύνδεση του λεκτικού με τον συντακτικό αναλυτή.

Έπειτα δηλώνονται οι μεταβλητές - μετρητές γραμμών, σωστών και λάθος λέξεων αλλά και warnings. Οι μεταβλητές δηλώνονται με extern αφού είναι δηλωμένες στον συντακτικό αναλυτή. Στην ουσία έχουμε μόνο μια μεταβλητή από την κάθε μια στο αρχείο bison και με το extern γίνεται χρήση τους σε εξωτερικά αρχεία. Έτσι επιτυγχάνουμε την κοινή χρήση των μεταβλητών και από τα 2 αρχεία και την εμφάνιση τους στο αρχείο bison.

Η σειρά των κανόνων είναι τέτοια ώστε στην αρχή να βρίσκονται οι πιο ειδικοί κανόνες και στο τέλος οι πιο γενικοί.

Επειδή ο λεκτικός αναλυτής όταν αναγνωρίσει έναν κανόνα δεν συνεχίζει την αναζήτηση στους επόμενους.

Ο κανόνας DELIMITER αναγνωρίζει έναν ή περισσότερους χαρακτήρες κενού ή tab και τους αγνοεί. Δεν επιστρέφει κανένα token γιατί τα whitespaces είναι ασήμαντα για τη γραμματική.

Ο κανόνας LEFT_PAR αναγνωρίζει τον χαρακτήρα (και επιστρέφει το token LEFT_PAR, που αντιστοιχεί στην αριστερή παρένθεση. Είναι βασικό δομικό σύμβολο της γλώσσας και χρησιμοποιείται για να αρχίσει κάθε input.

Ο κανόνας RIGHT_PAR αναγνωρίζει το) και επιστρέφει το token RIGHT_PAR, δηλαδή δεξιά παρένθεση. Χρησιμοποιείται για να ολοκληρώνει εκφράσεις, γεγονότα κτλ.

Ο κανόνας OPERATOR αναγνωρίζει αριθμητικούς τελεστές (+, -, *, /) και επιστρέφει το token OPERATOR. Παράλληλα, αυξάνει τον μετρητή correctWords.

Ο κανόνας ISON αναγνωρίζει τον τελεστή σύγκρισης =, και επιστρέφει το token ISON. Καταγράφεται ως σωστή λέξη.

Ο κανόνας ISON_WARN αναγνωρίζει το input ==. Θεωρούμε πως ο χρήστης ήθελε να δώσει =, οπότε επιστρέφει ISON, και εκτυπώνει προειδοποίηση στον χρήστη ότι έβαλε παραπάνω =. Τέλος αυξάνεται ο μετρητής warnings.

Ο κανόνας VELOS αναγνωρίζει το ->, και επιστρέφει το token VELOS. Αυξάνεται ο μετρητής correctWords. Το -> χρησιμοποιείται στη γλώσσα για το defrule.

Ο κανόνας VELOS_WARN εντοπίζει τα λάθη -->, > και - >, τα οποία είναι συχνά τυπογραφικά λάθη του ->. Προειδοποιεί τον χρήστη, επιστρέφει VELOS και αυξάνει το μετρητή warning.

Ο κανόνας INTCONST αναγνωρίζει προσημασμένους ή μη ακέραιους αριθμούς, είτε το 0 μόνο του. Επιστρέφει το token INTCONST. Αυξάνεται ο μετρητής correctWords.

Ο κανόνας FLOAT αναγνωρίζει αριθμούς κινητής υποδιαστολής με ή χωρίς εκθέτες. Επιστρέφει το token FLOAT. Αυξάνεται ο μετρητής correctWords.

Ο κανόνας VARIABLE αναγνωρίζει μεταβλητές και επιστρέφει VARIABLE, το οποίο χρησιμοποιείται σε bind, test. Αυξάνεται ο μετρητής correctWords.

Ο κανόνας COMMENTS αναγνωρίζει τα σχόλια και τα αγνοεί χωρίς να επιστρέφει κάποιο token.

Ο κανόνας DEFFACTS αναγνωρίζει το keyword deffacts και επιστρέφει το token DEFFACTS. Αυξάνεται ο μετρητής correctWords.

Ο κανόνας DEFFACTS_WARN εντοπίζει τα ορθογραφικά λάθη defacts, defact, deffact και προειδοποιεί τον χρήστη. Αυξάνεται ο μετρητής warnings και επιστρέφει DEFFACTS.

Ο κανόνας DEFRULE αναγνωρίζει τη λέξη defrule και επιστρέφει DEFRULE. Αυξάνεται ο μετρητής correctWords.

Ο κανόνας DEFRULE_WARN αναγνωρίζει τα ορθογραφικά λάθη derule, defrul, derulo και αυξάνεται ο μετρητής correctWords. Επιστρέφει DEFRULE αφού πρώτα εμφανιστεί η αντίστοιχη προειδοποίηση .

Ο κανόνας TEST αναγνωρίζει τη keyword λέξη test και επιστρέφει το token TEST. Αυξάνεται ο μετρητής correctWords.

Ο κανόνας TEST_WARN ανιχνεύει τα ορθογραφικά λάθη tst, ttest, testt και τα καταγράφει ως warning. Επιστρέφει κανονικά TEST.

Ο κανόνας PRINTOUT αναγνωρίζει το printout t και επιστρέφει το token PRINTOUT. Αυξάνεται ο μετρητής correctWords.

Ο κανόνας PRINTOUT_WARN καλύπτει τα ορθογραφικά λάθη printoutt, prntout, printout (χωρίς το t) και prntout t. Αυξάνεται ο μετρητής warnings, προειδοποιεί και επιστρέφει PRINTOUT.

Ο κανόνας READ αναγνωρίζει τη λέξη read, αυξάνεται ο μετρητής correctWords και επιστρέφει READ.

Ο κανόνας READ_WARN αναγνωρίζει τα ορθογραφικά λάθη readd, red, resd, και δίνει warning, επιστρέφει READ.

Ο κανόνας BIND αναγνωρίζει τη λέξη bind και επιστρέφει BIND. Αυξάνεται ο μετρητής correctWords.

Ο κανόνας BIND_WARN ανιχνεύει τα ορθογραφικά λάθη bindd, bbind, bnd, bbnd και δίνει προειδοποίηση. Αυξάνεται ο μετρητής warnings και επιστρέφει BIND.

Ο κανόνας FACT_RULE_NAME αναγνωρίζει τα ονόματα ορισμών και στοιχείων γεγονότων. Αυξάνεται ο μετρητής correctWords. Επιστρέφει FACT_RULE_NAME.

Ο κανόνας STRING αναγνωρίζει αλφαριθμητικά εντός εισαγωγικών ("..."), και επιτρέπει escape χαρακτήρες. Αυξάνεται ο μετρητής correctWords. Επιστρέφει το token STRING.

Ο κανόνας ERROR ενεργοποιείται για κάθε άγνωστο σύμβολο. Ο αναλυτής περνάει σε κατάσταση error, και αυξάνεται ο μετρητής wrongWords.

Ο κανόνας NEWLINE ανιχνεύει αλλαγή γραμμής και αυξάνει το line, ώστε να ξέρουμε σε ποια γραμμή εμφανίζονται λάθη.

Ο σκοπός της κατάστασης <error> είναι να απορρίπτει το λανθασμένο token μέχρι να λάβει κάποιον διαχωριστή που σημαίνει ότι έχει τελειώσει το εκάστοτε token και επιστρέφουμε στην εκτέλεση του προγράμματος με το BEGIN(0), αυξάνοντας τον μετρητή wrongWords.

Τεκμηρίωση του Συντακτικού Αναλυτή και της σχεδιάσής του

Ο συντακτικός αναλυτής αναγνωρίζει εκφράσεις αποτελούμενες από tokens με την βοήθεια τερματικών (token) και μη-τερματικών συμβόλων. Στην σχεδίαση μας θεωρούμε ως έγκυρες εκφράσεις τις εντολές `deffacts`, `defrule`, `test` και `bind`. Οποιαδήποτε άλλη έκφραση (π.χ. αριθμητική πράξη, σύγκριση, εντολή `printout` ή έκφραση μη υπαρκτή στη γλώσσα) θεωρείται συντακτικό λάθος.

Στην αρχή του προγράμματος έχουμε δηλώσει τις απαραίτητες βιβλιοθήκες, τις μεταβλητές-μετρητές που χρησιμοποιεί ο Συντακτικός Αναλυτής και που κάνει `extern` ο Λεκτικός Αναλυτής και τις συναρτήσεις `yylex` και `yycerror`.

Στο πεδίο των ορισμών αρχικά έχουμε ορίσει τα tokens που θα δεχθεί ο κώδικας `bison` από τον Λεκτικό Αναλυτή. Επίσης έχουμε ορίσει το `program`, ως εναρκτήριο μη-τερματικό, δηλαδή τον κανόνα που θα ξεκινήσει ο Συντακτικός Αναλυτής μας.

Ο κανόνας `program` περιγράφει αναδρομικά τον κατάλογο των εκφράσεων που αναγνωρίζονται από τον συντακτικό αναλυτή: `deffacts`, `defrule`, `test`, `bind`, και `error` για περιπτώσεις λάθους σύνταξης. Τα πρώτα 4 είναι μη-τερματικά σύμβολα, δηλαδή κανόνες που θα αναλυθούν αμέσως μετά, ενώ το τελευταίο (`error`) είναι ειδικό ενσωματωμένο token του `Bison`, το οποίο καλεί την συνάρτηση `yycerror` και επιτρέπει την συνέχεια της συντακτικής ανάλυσης παρά την ύπαρξη συντακτικού λάθους. Χωρίς τον κανόνα `program error` η ανάλυση θα σταματούσε στο πρώτο συντακτικό λάθος. Οι έγκυρες εκφράσεις εμφανίζουν την εντολή που αναγνωρίστηκε μαζί με την γραμμή της και αυξάνεται ο μετρητής `correctWords`.

Ο κανόνας `deffacts` αναγνωρίζει ορισμούς γεγονότων, η έκφραση των οποίων είναι: αριστερή παρένθεση, `DEFFACTS` που είναι το τερματικό που αφορά το keyword "`deffacts`", `FACT_RULE_NAME` που αντιπροσωπεύει ένα έγκυρο όνομα για τον ορισμό γεγονότων, μια σειρά από γεγονότα όπως περιγράφονται από τον κανόνα `fact_list` και τέλος μία δεξιά παρένθεση. Αν λείπει η αριστερή παρένθεση, προβλέπεται εναλλακτικός κανόνας με `warning`, που εκτυπώνει προειδοποίηση και αυξάνει τον μετρητή `warnings`.

Ο κανόνας `fact_list` υλοποιεί μια λίστα γεγονότων, όπου κάθε γεγονός περικλείεται σε παρενθέσεις. Το token `LEFT_PAR` και `RIGHT_PAR` οριοθετούν κάθε γεγονός, ενώ ο κανόνας `elements` καθορίζει τι μπορεί να περιέχει το γεγονός. Συγκεκριμένα θεωρούμε ότι τα γεγονότα δέχονται ακέραιους, μεταβλητές, μαθηματικές πράξεις και strings. Σε κάθε νέα αναγνώριση γεγονότος, γίνεται εκτύπωση "Γεγονός".

Ο κανόνας `elements` περιλαμβάνει κάθε πιθανό στοιχείο ενός γεγονότος: τον κανόνα `expr` (ακέραιες τιμές, μεταβλητές, πράξεις), `FACT_RULE_NAME` (ονόματα), και `STRING` που είναι αλφαριθμητικά μέσα σε διπλά εισαγωγικά.

Ο κανόνας `defrule` αναγνωρίζει ορισμούς κανόνων. Η έκφραση συντάσσεται ως εξής: αριστερή παρένθεση, το τερματικό `DEFRULE`, ένα όνομα για τον κανόνα `FACT_RULE_NAME`, μια σειρά γεγονότων που αναγνωρίζονται με τον κανόνα `fact_list`, μία δομή ελέγχου `test`, το token `VELOS`, μία ή περισσότερες εντολές `printout` και τέλος την δεξιά παρένθεση. Αν λείπει η αριστερή παρένθεση,

προβλέπεται εναλλακτικός κανόνας με `warning`, που εκτυπώνει προειδοποίηση και αυξάνει τον μετρητή `warnings`.

Ο κανόνας `test` περιέχει ελέγχους ισότητας μέσω της λέξης-κλειδί `TEST`, η οποία συνοδεύεται από τον κανόνα `equal`. Η πλήρης μορφή είναι (`test (= expr expr)`), με tokens `LEFT_PAR`, `TEST`, `equal` (το `=`) `RIGHT_PAR`. Αν λείπει η `(`, υπάρχει `warning`, αυξάνεται το `warnings` και ενημερώνεται ο χρήστης.

Ο κανόνας `printout` αφορά εντολές τύπου (`printout t ("Hello")`). Περιλαμβάνει τα tokens `PRINTOUT` (που ορίζεται ως `"printout t"`), `LEFT_PAR`, `RIGHT_PAR`, και επαναληπτικά tokens που ταιριάζουν με το `fact_list`. Μετά το token `PRINTOUT` μπορούν να ακολουθούν ένα ή περισσότερα γεγονότα. Αν λείπει η παρένθεση στην αρχή, ενεργοποιείται ειδικός κανόνας που εκτυπώνει `warning`.

Ο κανόνας `mathoperation` εκφράζει πράξεις της μορφής (`+ expr expr ...`), χρησιμοποιώντας το token `OPERATOR` (που καλύπτει `+`, `-`, `*`, `/`), και τους `expr` και `expr_list`. Περιβάλλεται από `LEFT_PAR` και `RIGHT_PAR`. Μπορεί να περιέχει δύο ή περισσότερες μαθηματικές εκφράσεις καθώς και εμφολευμένες μαθηματικές εκφράσεις.

Ο κανόνας `expr` δηλώνει ότι μια έκφραση μπορεί να είναι ένας ακέραιος αριθμός (`INTCONST`), μια μεταβλητή (`VARIABLE`, που πρέπει να ξεκινάει με `?`), ή μια μαθηματική έκφραση (`mathoperation`). Τα tokens που χρησιμοποιούνται είναι `INTCONST`, `VARIABLE`, και ολόκληρος ο κανόνας `mathoperation`.

Ο κανόνας `expr_list` ορίζει μια λίστα από αριθμητικές εκφράσεις, επιτρέποντας και μια σειρά από αριθμητικές εκφράσεις με την βοήθεια της αναδρομής. Δεν εμπλέκει νέα tokens, αλλά βασίζεται στον κανόνα `expr`.

Ο κανόνας `equal` καθορίζει τη δομή σύγκρισης τύπου (`= expr expr`). Η σύνταξη είναι: `LEFT_PAR`, `ISON`, `expr`, `expr` και `RIGHT_PAR`. Αν το `=` λείπει (δηλαδή έχει γραφεί (`expr expr`)), τότε ενεργοποιείται ειδικός κανόνας με `warning` και γίνεται η κατάλληλη προσμέτρηση.

Ο κανόνας `read` αναγνωρίζει τη συνάρτηση εισόδου (`read`), με tokens `LEFT_PAR`, `READ`, `RIGHT_PAR`. Αν λείπει η παρένθεση αναγνωρίζει την έκφραση αλλά εμφανίζεται μήνυμα `warning` και αυξάνεται ο μετρητής `warnings`. Το `READ` εδώ αντιπροσωπεύει την εντολή λήψης τιμής από τον χρήστη.

Τέλος ο κανόνας `bind` είναι υπεύθυνος για την ανάθεση τιμών σε μεταβλητές. Χρησιμοποιεί το token `LEFT_PAR`, `BIND` (η λέξη `"bind"`), ένα `VARIABLE` ή μία συνάρτηση `read` και κλείνει με `RIGHT_PAR`. Μόλις αναγνωρίζεται εμφανίζεται “Ανάθεση τιμής”. Αν σε κάποια περίπτωση ξεχαστεί το `LEFT_PAR` (“`(`”) τότε αναγνωρίζεται η έκφραση αλλά εμφανίζει `warning` και αυξάνει τον μετρητή `warnings`.

Η `yyperror` καλείται από το τερματικό σύμβολο `error` του κανόνα `program error` και εμφανίζει μήνυμα συντακτικού λάθους.

Τέλος, στη κύρια συνάρτηση (`main`) εκτελείται η `yyparse` η οποία εκκινεί την συντακτική ανάλυση. Επίσης η `main` λειτουργεί ως σημείο συγκέντρωσης ποσοτικών

στοιχείων που σχετίζονται με την ορθότητα της ανάλυσης. Οι μεταβλητές `correctExprs`, `incorrectExprs`, `correctWords`, `incorrectWords` παρέχουν στατιστικά για το τρέξιμο του προγράμματος. Η σύγκριση του αποτελέσματος με τους παραπάνω μετρητές επιτρέπει την διάκριση ανάμεσα σε ολική επιτυχία, αποτυχία λόγω συντακτικού λάθους και μερική επιτυχία (warnings).

Τρόπος μεταγλώττισης και τρεξίματος

Μαζί με τα αρχεία του λεκτικού και του συντακτικού αναλυτή παρέχεται και ένα `makefile` για την μεταγλώττιση τους. Εκτελώντας **make** στον κατάλογο στον οποίο έχει αποσυμπίεστεί το παραδοτέο, δημιουργείται το εκτελέσιμο **bison_code**. Μπορεί να εκτελεστεί είτε μόνο του (`./bison_code`) ώστε να δέχεται είσοδο από το πληκτρολόγιο ή με ανακατεύθυνση της εισόδου και της εξόδου (π.χ. `./bison_code < in.txt > out.txt`) ώστε να δέχεται είσοδο από αρχείο εισόδου και να γράφει την έξοδο σε κάποιο αντίστοιχο αρχείο.

Οι εντολές που εκτελούνται κατά το `make` είναι :

```
bison -o teliko_y.c teliko.y -d
flex -o teliko_l.c teliko.l
gcc -o bison_code teliko_l.c teliko_y.c -lfl -lm
```

Η πρώτη εντολή παίρνει το αρχείο της γραμματικής **teliko.y** και παράγει το κώδικα C του parser **teliko_y.c** και το header αρχείο **teliko_y.h** με την επιλογή `-d` για την σύνδεση του λεκτικού με τον συντακτικό.

Η δεύτερη εντολή παίρνει το αρχείο Flex **teliko.l** και δημιουργεί τον κώδικα C για τον αναλυτή των λεξικών μονάδων.

Τέλος η τρίτη εντολή μεταγλωττίζει τα 2 παραπάνω C αρχεία στο τελικό εκτελέσιμο πρόγραμμα **bison_code**. Η επιλογή `-lfl` συνδέει το πρόγραμμα με την βιβλιοθήκη του Flex.

Εξαντλητικοί έλεγχοι

Παρακάτω παρουσιάζεται το περιεχόμενο του αρχείου εισόδου `in.txt` σε αντιδιαστολή με την έξοδο:

1	;;ορισμός των γεγονότων του προβλήματος (defacts static-facts ;;; food declarations (food-at 4 2) (food-at 5 2))	Γεγονός Γεγονός Βρέθηκε Ορισμός Γεγονότων στη γραμμή 5
2	;;κανόνας όπου ο χαρακτήρας κινείται βόρεια ;;μόνο αν υπάρχει τροφή εκεί (defrule move_UP (pacman_at ?x ?y) (food_at ?z ?y)	Γεγονός Γεγονός Μαθηματική Έκφραση Σύγκριση Έλεγχος Γεγονός

	<pre>(test (= ?z (- ?x 1))) -> ;;; prints just a message (printout t ("pacman has reached food")))</pre>	Συνάρτηση printout Βρέθηκε Ορισμός Κανόνα στη γραμμή 15
3	<pre>(defrule move-up (cube A on ?something) (cube B on A) (number_of_cubes ?num) (test(= ?num 2)) -> (printout t (?something " is under A") ("A is under B") ("there are 2 cubes"))))</pre>	Γεγονός Γεγονός Γεγονός Σύγκριση Έλεγχος Γεγονός Γεγονός Γεγονός Συνάρτηση printout Βρέθηκε Ορισμός Κανόνα στη γραμμή 23
4	<pre>(defrule basic (initial_fact) (test (= 2 2)) -> (printout t ("Hello, world!")))</pre>	Γεγονός Σύγκριση Έλεγχος Γεγονός Συνάρτηση printout Βρέθηκε Ορισμός Κανόνα στη γραμμή 29
5	<pre>(defrule check_count (number_of_cubes ?num) (test (= ?num 1)) -> (printout t ("There are more than 1 cubes"))))</pre>	Γεγονός Σύγκριση Έλεγχος Γεγονός Συνάρτηση printout Βρέθηκε Ορισμός Κανόνα στη γραμμή 35
6	<pre>(deffacts robot_world (robot_position x 5 y 10) (battery_level 85) (obstacle front no) (obstacle left yes) (obstacle right no))</pre>	Γεγονός Γεγονός Γεγονός Γεγονός Γεγονός Βρέθηκε Ορισμός Γεγονότων στη γραμμή 43
7	<pre>(defrule move_forward (robot_position x ?x y ?y) (battery_level ?level) (test (= ?level 50)) -> (printout t ("Moving robot forward"))))</pre>	Γεγονός Γεγονός Σύγκριση Έλεγχος Γεγονός Συνάρτηση printout Βρέθηκε Ορισμός Κανόνα στη γραμμή 51
8	<pre>(defrule energy_check (battery_level ?b)</pre>	Γεγονός Σύγκριση

	<pre>(test (= ?b 80)) -> (printout t ("Battery is between 20 and 80")))</pre>	Έλεγχος Γεγονός Συνάρτηση printout Βρέθηκε Ορισμός Κανόνα στη γραμμή 57
9	<pre>(defrule greet_user (user_name ?name) (test (= ?name 2)) -> (printout t ("Hello, " ?name "!")))</pre>	Γεγονός Σύγκριση Έλεγχος Γεγονός Συνάρτηση printout Βρέθηκε Ορισμός Κανόνα στη γραμμή 63
10	<pre>(test (= 2 2))</pre>	Σύγκριση Έλεγχος Βρέθηκε Test Function στη γραμμή 65
11	<pre>(test (= ?num 2))</pre>	Σύγκριση Έλεγχος Βρέθηκε Test Function στη γραμμή 66
12	<pre>(test (= ?x 100))</pre>	Σύγκριση Έλεγχος Βρέθηκε Test Function στη γραμμή 67
13	<pre>(test (= ?level 50))</pre>	Σύγκριση Έλεγχος Βρέθηκε Test Function στη γραμμή 68
14	<pre>(test (= ?battery 80))</pre>	Σύγκριση Έλεγχος Βρέθηκε Test Function στη γραμμή 69
15	<pre>(test (= (+ 1 1) 2))</pre>	Μαθηματική Έκφραση Σύγκριση Έλεγχος Βρέθηκε Test Function στη γραμμή 70
16	<pre>(test (= (* ?a 2) ?b))</pre>	Μαθηματική Έκφραση Σύγκριση Έλεγχος

		Βρέθηκε Test Function στη γραμμή 71
17	(test (= (- ?value 10) 0))	Μαθηματική Έκφραση Σύγκριση Έλεγχος Βρέθηκε Test Function στη γραμμή 72
18	(test (= (/ ?total ?count) 5))	Μαθηματική Έκφραση Σύγκριση Έλεγχος Βρέθηκε Test Function στη γραμμή 73
19	(test (= (* ?a 2) 5))	Μαθηματική Έκφραση Σύγκριση Έλεγχος Βρέθηκε Test Function στη γραμμή 74
20	(bind ?x (read))	Ανάγνωση Ανάθεση τιμής Βρέθηκε Bind Function στη γραμμή 75
21	(bind ?var 15)	Ανάθεση τιμής Βρέθηκε Bind Function στη γραμμή 76
22	(bind ?x (+ 4 5))	Μαθηματική Έκφραση Ανάθεση τιμής Βρέθηκε Bind Function στη γραμμή 77
23	(bind ?x (* 4 100))	Μαθηματική Έκφραση Ανάθεση τιμής Βρέθηκε Bind Function στη γραμμή 78
24	(bind ?a(+ 23 15))	Μαθηματική Έκφραση Ανάθεση τιμής Βρέθηκε Bind Function στη γραμμή 79

25	<code>(bind ?cold (- 10 10))</code>	Μαθηματική Έκφραση Ανάθεση τιμής Βρέθηκε Bind Function στη γραμμή 80
26	<code>(bind ?counter 0)</code>	Ανάθεση τιμής Βρέθηκε Bind Function στη γραμμή 81
27	<code>(bind ?sum (+ ?a ?b ?c))</code>	Μαθηματική Έκφραση Ανάθεση τιμής Βρέθηκε Bind Function στη γραμμή 82
28	<code>(bind ?result (/ 100 4))</code>	Μαθηματική Έκφραση Ανάθεση τιμής Βρέθηκε Bind Function στη γραμμή 83
29	<code>(bind ?area (* ?width ?height))</code>	Μαθηματική Έκφραση Ανάθεση τιμής Βρέθηκε Bind Function στη γραμμή 84

Τεκμηρίωση in.txt

- Ορισμός Γεγονότων (def facts static-facts) Στις γραμμές 1–4 εντοπίζεται ο ορισμός γεγονότων με το def facts static-facts· μέσα σε αυτό δηλώνονται δύο γεγονότα (food-at 4 2) και (food-at 5 2) μόλις κλείσει η παρένθεση του def facts, ολοκληρώνεται ορισμός των στατικών γεγονότων.
- Κανόνας (def rule move_UP)
 - Γεγονότα: Αναγνωρίζονται τα (pacman_at ?x ?y) και (food_at ?z ?y) με μεταβλητές.
 - Μαθηματική Έκφραση: Υπολογίζεται το (- ?x 1).
 - Σύγκριση: Γίνεται σύγκριση = ?z (- ?x 1).
 - Έλεγχος: Όλα τα παραπάνω περνάνε σε έλεγχο μέσω (test (= ?z (- ?x 1))).
 - Γεγονός: ("pacman has reached food").
 - Συνάρτηση printout: (printout t ("pacman has reached food"))
 - Μόλις κλείσει η παρένθεση του def rule, ολοκληρώνεται ο ορισμός του κανόνα.
- Κανόνας (def rule move-up).
 - Γεγονότα: (cube A on ?something), (cube B on A), (number_of_cubes ?num).
 - Σύγκριση και Έλεγχος: (test (= ?num 2)).
 - Γεγονότα και Συνάρτηση printout: printout t (?something " is under A") ("A is under B") ("there are 2 cubes").

4. Μόλις κλείσει η παρένθεση του `defrule`, ολοκληρώνεται ο ορισμός του κανόνα.
4. Κανόνας (`defrule basic`).
 1. Γεγονός: (`initial_fact`).
 2. Σύγκριση και Έλεγχος: (`test (= 2 2)`).
 3. Γεγονός και Συνάρτηση `printout`: `printout t ("Hello, world!")`.
 4. Μόλις κλείσει η παρένθεση του `defrule`, ολοκληρώνεται ο ορισμός του κανόνα.
5. Κανόνας (`defrule check_count`).
 1. Γεγονός: (`number_of_cubes ?num`).
 2. Σύγκριση και Έλεγχος: (`test (= ?num 1)`).
 3. Γεγονός και Συνάρτηση `printout`: `printout t ("There are more than 1 cubes")`.
 4. Μόλις κλείσει η παρένθεση του `defrule`, ολοκληρώνεται ο ορισμός του κανόνα.
6. Ορισμός Γεγονότων (`deffacts robot_world`).
 1. Ορίζονται γεγονότα: (`robot_position x 5 y 10`), (`battery_level 85`), (`obstacle front no`), (`obstacle left yes`), (`obstacle right no`)
 2. Μόλις κλείσει η παρένθεση του `deffacts`, ολοκληρώνεται ο ορισμός του ορισμός γεγονότων.
7. Κανόνας (`defrule move_forward`).
 1. Γεγονότα: (`robot_position x ?x y ?y`), (`battery_level ?level`).
 2. Σύγκριση και Έλεγχος: (`test (= ?level 50)`).
 3. Γεγονός και Συνάρτηση `printout`: `printout t ("Moving robot forward")`.
 4. Μόλις κλείσει η παρένθεση του `defrule`, ολοκληρώνεται ο ορισμός του κανόνα.
8. Κανόνας (`defrule energy_check`).
 1. Γεγονός: (`battery_level ?b`).
 2. Σύγκριση και Έλεγχος: (`test (= ?b 80)`).
 3. Γεγονός και Συνάρτηση `printout`: `printout t ("Battery is between 20 and 80")`.
 4. Μόλις κλείσει η παρένθεση του `defrule`, ολοκληρώνεται ο ορισμός του κανόνα.
9. Κανόνας (`defrule greet_user`).
 1. Γεγονός: (`user_name ?name`).
 2. Σύγκριση και Έλεγχος: (`test (= ?name 2)`).
 3. Γεγονός και Συνάρτηση `printout`: `printout t ("Hello, " ?name "!")`.
 4. Μόλις κλείσει η παρένθεση του `defrule`, ολοκληρώνεται ο ορισμός του κανόνα.
10. Δοκιμαστικές συναρτήσεις (`test`) εκτός κανόνων.
 1. Απλή σύγκριση: (`test (= 2 2)`), (`test (= ?num 2)`), (`test (= ?x 100)`), (`test (= ?level 50)`), (`test (= ?battery 80)`), (`test (= (+ 1 1) 2)`), (`test (= (* ?a 2) ?b)`), (`test (= (- ?value 10) 0)`), (`test (= (/ ?total ?count) 5)`), (`test (= (* ?a 2) 5)`).
 2. Μαθηματικές εκφράσεις με Σύγκριση και Έλεγχο: (`(= (+ 1 1) 2)`), (`(= (* ?a 2) ?b)`), (`(= (- ?value 10) 0)`), (`(= (/ ?total ?count) 5)`), (`(= (* ?a 2) 5)`) και (`(= (* ?a 2) 5)`).
11. Συναρτήσεις ανάθεσης (`bind`).
 1. Ανάγνωση και Ανάθεση: (`bind ?x (read)`).
 2. Απλή ανάθεση: (`bind ?var 15`), (`bind ?counter 0`).

3. Μαθηματική Έκφραση και Ανάθεση: (bind ?x (+ 4 5)), (bind ?x (* 4 100)), (bind ?a (+ 23 15)), (bind ?cold (- 10 10)), (bind ?sum (+ ?a ?b ?c)), (bind ?result (/ 100 4)), (bind ?area (* ?width ?height)).

Παρακάτω παρουσιάζεται το περιεχόμενο του αρχείου εισόδου με λάθος εκφράσεις in_wrong.txt σε αντιδιαστολή με την έξοδο:

1	(test (== 2 2)) ;; ISON_WARN	Warning: Έβαλες 2 ίσον. Σύγκριση Έλεγχος Βρέθηκε Test Function στη γραμμή 2
2	(defacts my_facts (object a))	Βρέθηκε Ορισμός Γεγονότων στη γραμμή 7 Warning: Έγραψες λάθος την συνάρτηση deffacts. Γεγονός Βρέθηκε Ορισμός Γεγονότων στη γραμμή 11
3	(defact my_facts3 (object c))	Warning: Έγραψες λάθος την συνάρτηση deffacts. Γεγονός Βρέθηκε Ορισμός Γεγονότων στη γραμμή 15
4	(derule move_left (position ?x ?y) (tst (== ?x 5)) ;; TEST_WARN + ISON_WARN --> (prntout t ("Moving left")) ;; PRINTOUT_WARN)	Warning: Έγραψες λάθος την συνάρτηση defrule. Γεγονός Warning: Έγραψες λάθος την συνάρτηση test. Warning: Έβαλες 2 ίσον. Σύγκριση Έλεγχος Warning: Έγραψες λάθος το βέλος. Warning: Έγραψες λάθος την συνάρτηση prntout. Γεγονός Συνάρτηση prntout Βρέθηκε Ορισμός Κανόνα στη γραμμή 23
5	(defrul move_right (position ?x ?y)	Warning: Έγραψες λάθος την συνάρτηση defrule.

	<pre> (ttest (= ?y 3)) ;; TEST_WARN --> (printoutt ("Moving right")) ;; PRINTOUT_WARN) </pre>	Γεγονός Warning: Έγραψες λάθος την συνάρτηση test. Σύγκριση Έλεγχος Warning: Έγραψες λάθος το βέλος. Warning: Έγραψες λάθος την συνάρτηση printout. Γεγονός Συνάρτηση printout Βρέθηκε Ορισμός Κανόνα στη γραμμή 30
6	<pre> (derulo move_down (position ?x ?y) (testt (= ?y 0)) ;; TEST_WARN --> ;; VELOS_WARN (printout t ("Moving down")) ;; PRINTOUT_WARN (the correct but also in the warn list!)) </pre>	Warning: Έγραψες λάθος την συνάρτηση defrule. Γεγονός Warning: Έγραψες λάθος την συνάρτηση test. Σύγκριση Έλεγχος Warning: Έγραψες λάθος το βέλος. Γεγονός Συνάρτηση printout Βρέθηκε Ορισμός Κανόνα στη γραμμή 37
7	<pre> ;; READ_WARN (bindd ?input1 (readd)) ;; BIND_WARN + READ_WARN </pre>	Warning: Έγραψες λάθος την συνάρτηση bind. Warning: Έγραψες λάθος την συνάρτηση read. Ανάγνωση Ανάθεση τιμής Βρέθηκε Bind Function στη γραμμή 40
8	<pre> (bindd ?input2 (red)) ;; BIND_WARN + READ_WARN </pre>	Warning: Έγραψες λάθος την συνάρτηση bind. Warning: Έγραψες λάθος την συνάρτηση read. Ανάγνωση Ανάθεση τιμής Βρέθηκε Bind Function στη γραμμή 41
9	<pre> (bindd ?input3 (resd)) ;; BIND_WARN + READ_WARN </pre>	Warning: Έγραψες λάθος την συνάρτηση bind. Warning: Έγραψες λάθος την συνάρτηση read. Ανάγνωση Ανάθεση τιμής

		Βρέθηκε Bind Function στη γραμμή 42
10	<code>;; BIND_WARN (remaining variations) (bbnd ?counter 0)</code>	Warning: Έγραψες λάθος την συνάρτηση bind. Ανάθεση τιμής Βρέθηκε Bind Function στη γραμμή 45
11	<code>(bnd ?value (+ 2 3))</code>	Warning: Έγραψες λάθος την συνάρτηση bind. Μαθηματική Έκφραση Ανάθεση τιμής Βρέθηκε Bind Function στη γραμμή 46
12	<code>;; VELOS_WARN (all variants) (defrule move_up (position ?x ?y) (test (= ?x 1)) --> (printout t ("Using -->")) ;; VELOS_WARN)</code>	Γεγονός Σύγκριση Έλεγχος Warning: Έγραψες λάθος το βέλος. Γεγονός Συνάρτηση printout Βρέθηκε Ορισμός Κανόνα στη γραμμή 54
13	<code>(defrule move_diag (position ?x ?y) (test (= ?y 1)) > ;; VELOS_WARN (printout t ("Using >")))</code>	Γεγονός Σύγκριση Έλεγχος Warning: Έγραψες λάθος το βέλος. Γεγονός Συνάρτηση printout Βρέθηκε Ορισμός Κανόνα στη γραμμή 61
14	<code>(defrule move_random (position ?x ?y) (test (= ?x ?y)) - > ;; VELOS_WARN (printout t ("Using - >")))</code>	Γεγονός Σύγκριση Έλεγχος Warning: Έγραψες λάθος το βέλος. Γεγονός Συνάρτηση printout Βρέθηκε Ορισμός Κανόνα στη γραμμή 68
15	<code>(printout t ("Hello"))</code>	Error: Συντακτικό λάθος στη γραμμή 70: syntax error
16	<code>(bindd ?input3 (resd))</code>	Warning: Έγραψες λάθος την συνάρτηση bind. Warning: Έγραψες λάθος την συνάρτηση read.

		Ανάγνωση Ανάθεση τιμής Βρέθηκε Bind Function στη γραμμή 71
17	(+ 1 1)	Error: Συντακτικό λάθος στη γραμμή 72: syntax error
18	{@@#}	Λάθος στην γραμμή: 74

Τεκμηρίωση in_wrong.txt

Ο συντακτικός αναλυτής αφού αναγνωρίσει κάποιο συντακτικό λάθος, μετά θα πρέπει να αναγνωρίσει τρία έγκυρα tokens για να συνεχίσει την εκτέλεση του.

1. Βγάζει warning λόγω του == (ISON_WARN), επειδή ο τελεστής σύγκρισης πρέπει να είναι =. Παρόλα αυτά, ο parser καταλαβαίνει ότι πρόκειται για σύγκριση και συνεχίζει κανονικά την ανάλυση. Αναγνωρίζει το test ως Έλεγχος και εμφανίζει "Βρέθηκε Test Function".
2. Βγάζει warning γιατί χρησιμοποιείται το defacts αντί για το σωστό deffacts (DEFFACTS_WARN). Ο parser αναγνωρίζει ότι πρόκειται για Ορισμό Γεγονότων και ολοκληρώνει σωστά την αναγνώριση με "Βρέθηκε Ορισμός Γεγονότων".
3. Βγάζει warning λόγω της χρήσης του defact αντί για deffacts (DEFFACTS_WARN). Παρόλα αυτά, ο parser αναγνωρίζει την εντολή ως Ορισμό Γεγονότων και εμφανίζει "Βρέθηκε Ορισμός Γεγονότων".
4. Βγάζει πολλά warnings:
 - DEFRULE_WARN από derule.
 - TEST_WARN από tst.
 - ISON_WARN από ==.
 - VELOS_WARN από -->.
 - PRINTOUT_WARN από prntout.

Παρόλα αυτά, ο parser αναγνωρίζει ότι είναι Κανόνας (Rule) και εμφανίζει "Βρέθηκε Ορισμός Κανόνα".

5. Βγάζει warnings:
 - DEFRULE_WARN από defrul.
 - TEST_WARN από ttest.
 - VELOS_WARN από -->.

- PRINTOUT_WARN από printoutt.

Ο parser αναγνωρίζει τον Κανόνα κανονικά και ολοκληρώνει με "Βρέθηκε Ορισμός Κανόνα".

6. Βγάζει warnings:

- DEFRULE_WARN από derulo.
- TEST_WARN από testt.
- VELOS_WARN από -->.

Ο parser αναγνωρίζει τον Κανόνα κανονικά και εμφανίζει "Βρέθηκε Ορισμός Κανόνα".

7. Βγάζει warnings:

- BIND_WARN από bindd.
- READ_WARN από readd.

Παρότι υπάρχουν warnings, ο parser αναγνωρίζει σωστά την εντολή ως Bind Function και ως Ανάγνωση.

8. Βγάζει warnings:

- BIND_WARN από bindd.
- READ_WARN από red.

Ο parser αναγνωρίζει σωστά ως Bind Function + Ανάγνωση.

9. Βγάζει warnings:

- BIND_WARN από bindd.
- READ_WARN από resd.

Ο parser αναγνωρίζει σωστά ως Bind Function + Ανάγνωση.

10. Βγάζει warning:

- BIND_WARN από bbnd.

Αναγνωρίζεται σωστά ως Bind Function (Ανάθεση τιμής).

11. Βγάζει warning:

- BIND_WARN από bnd.

Η έκφραση (+ 2 3) αναγνωρίζεται ως Μαθηματική Έκφραση και όλη η δήλωση ολοκληρώνεται ως Bind Function.

12. Βγάζει warning:

- VELOS_WARN από -->.

Η σύγκριση στο test αναγνωρίζεται σωστά ως Έλεγχος. Ο Κανόνας ολοκληρώνεται και εμφανίζεται "Βρέθηκε Ορισμός Κανόνα".

13. Βγάζει warning:

- VELOS_WARN από >.

Η σύγκριση στο test αναγνωρίζεται σωστά. Ο parser αναγνωρίζει κανονικά τον Κανόνα.

14. Βγάζει warning:

- VELOS_WARN από - >.

Η σύγκριση στο test αναγνωρίζεται σωστά. Ο Κανόνας ολοκληρώνεται και εμφανίζεται "Βρέθηκε Ορισμός Κανόνα".

15. Ο bison βγάζει Error: Συντακτικό λάθος στη γραμμή 70: syntax error. Διότι στην υλοποίηση μας δεν δέχεται μια σκέτη συνάρτηση printout (printout t ("Hello")).

16. Ο bison βγάζει: Warning: Έγραψες λάθος την συνάρτηση bind.

Warning: Έγραψες λάθος την συνάρτηση read.

Ανάγνωση Ανάθεση τιμής

Βρέθηκε Bind Function. Διότι δεν έχει γραφτεί σωστά το bind και το read αλλά αναγνωρίζεται όλο ως ανάθεση τιμής.

17. Ο bison βγάζει: Error: Συντακτικό λάθος στη γραμμή 72: syntax error. Διότι στην υλοποίηση μας δεν δέχεται μια σκέτη μαθηματική πράξη (+ 1 1).

18. Ο Lex βγάζει: Λάθος στην γραμμή: 74. Καθώς κάνει match το "{" με τον κανόνα ERROR και παραμένει στην κατάσταση <error> μέχρι να ολοκληρωθεί το λάθος token, και επιστρέφει στην ροή του προγράμματος.

Αναφορές κώδικα

Ο κώδικας μεταγλωττίζεται και εκτελείται επιτυχώς χωρίς σφάλματα και όλα τα απαραίτητα αρχεία παράγονται κανονικά με το τελικό εκτελέσιμο να δημιουργείται χωρίς warnings ή errors κατά τη μεταγλώττιση. Οι μετρητές για σωστές και λανθασμένες λέξεις/εκφράσεις και τα warnings λειτουργούν όπως προβλέπεται από την εκφώνηση, και η συνολική συμπεριφορά του προγράμματος επιβεβαιώνει ότι όλα τα μέρη του συστήματος συνεργάζονται σωστά.

Αναλυτική αναφορά αρμοδιοτήτων ομάδας

1. Υλοποίηση κώδικα flex/bison: Συλλογικά.
2. Τεκμηρίωση της γραμματικής, του σχεδιασμού, και του κώδικα,: Μάθησης Ιωάννης-Ιάσων.
3. Παρουσίαση εξαντλητικών ελέγχων με σχολιασμένα αποτελέσματα και τελικά σχόλια: Τσαρτσίου Ερνίστ, Παναγιώτης Ξηρουδάκης.
4. Αναλυτικός σχολιασμός κώδικα για flex (αρχείο .l) και bison (αρχείο .y): Σοφινίδης Κωνσταντίνος, Λουκάς Στυλιανός.
5. Συντονιστής Ομάδας: Λουκάς Στυλιανός.

Κανένα μέλος δεν αποχώρησε κατά την διάρκεια του εξαμήνου.